Ders 19 Genişletilmiş Notu: Graf Veri Yapısı ve Minimum Yayılan Ağaç

Ders 19 Genişletilmiş Notu: Graf Veri Yapısı ve Minimum Yayılan Ağaç

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Graf: Doğrusal Olmayan Bir Veri Yapısı
- 2. Grafın Gerçek Hayattaki Kullanım Alanları
- 3. Graf Terminolojisi
- 4. Sembol Tablosu: Düğümleri Sayılarla Temsil Etmek
- 5. Grafın Saklanması: Komşuluk Matrisi
- 6. Yoğun ve Seyrek Graflar
- 7. Komşuluk Listesi ve C ile Gerçeklenmesi
- 8. Komşuluk Matrisi ile Komşuluk Listesinin Karşılaştırılması
- 9. Yönlü Graflar
- 10. Sembol Graf Örneği: Havaalanları
- 11. Minimum Yayılan Ağaç
- 12. Prim Algoritması
- 13. Kruskal Algoritması
- 14. Döngü Kontrolü: Etiket Dizisi
- 15. Kruskal Algoritmasının Taslağı
- 16. Çalışma Süreleri

1. Graf: Doğrusal Olmayan Bir Veri Yapısı

Hoca derse, şimdiye kadar gördüğümüz veri yapılarını hatırlatarak başladı. Dizi, bağlı liste (linked list), yığın (stack) ve kuyruk (queue) doğrusal (linear) veri yapılarıdır. Yığında ve kuyrukta bilgiler ardışık olarak bulunur. Bu yapıları dizi kullanarak da gerçekledik ve aslında verileri hep ardışık biçimde, bir dizinin içinde tuttuk. Yapılar arasındaki fark yalnızca erişim şeklidir: birinde son gelen ilk çıkar, diğerinde ilk gelen ilk çıkar. Ama her durumda bilgiler sıralı bir düzen içinde durur. Yığında "A buradadır, arkasında B vardır, onun arkasında C vardır" diyebiliriz. Bağlı listede de aynı şekilde sıralı (sequential) bir erişim söz konusudur.

Graf (graph) veri yapısı ise doğrusal değildir (non-linear). Grafta düğümler ve bu düğümler arasındaki bağlantılar vardır. Bilgilerin ardışık ya da sıralı olması diye bir şey yoktur. Aralarında ilişki olan bilgiler, bu ilişkiye göre birbirine bağlanır. Hoca, grafın günümüzde günlük hayatta en çok kullanılan veri yapılarından biri olduğunu söyledi.

2. Grafın Gerçek Hayattaki Kullanım Alanları

2.1 İnternet

Akla gelen ilk graf örneği internettir. İnternette düğümler router'lardır, yani ağ üzerinde verinin iletilmesini sağlayan fiziksel ya da sanal (virtual) cihazlardır. Bağlantılar ise bu cihazlar

arasındaki bağlantılardır. Kablolarla farklı router'ları fiziksel olarak birbirine bağlamış olursunuz. Hoca, internetin 1997-2021 yılları arasındaki gelişimini gösteren kısa bir görselleştirme videosu (Opte projesi) izletti ve internet grafının daha ayrıntılı simülasyonlarına sonradan bakılabileceğini söyledi. Hocanın vurguladığı nokta şuydu: hayatta farkına varmadan sürekli kullandığımız en temel graf yapısı internettir.

2.2 Haritalar

İkinci örnek Google Maps gibi harita uygulamalarıdır. Kesişim noktaları (intersection) grafın düğümleridir, aradaki yollar da bağlantılardır. "Kadıköy'den Beşiktaş'a nasıl giderim?" diye sorduğunuzda aslında bir graf üzerinde işlem yapılır. En kısa nasıl gidebilirim, en hızlı nasıl gidebilirim, oradan oraya yol var mı gibi soruların hepsi graf üzerindeki bilgilerin değerlendirilmesiyle cevaplanır.

Hoca örneği şöyle somutlaştırdı: Kadıköy'den Beşiktaş'a giderken Altunizade'den, Beylerbeyi'nden geçebilirsiniz. Kadıköy ile Altunizade arasındaki bağlantıda kaç kilometre olduğu, o anki trafiğe göre ne kadar sürdüğü, hangi taşıtlarla gidilebileceği gibi bilgiler tutulabilir. Bu sayede hangi taşıtlarla gidilebileceği, yolun kaç kilometre ve ne kadar süreceği, alternatif yolların neler olduğu görülebilir. Burada doğrusal bir bağlantı yoktur: Kadıköy'den Beşiktaş'a vapurla da gidebilirsiniz, karayoluyla da. Bir hiyerarşi yoktur, farklı yollar ve farklı bağlantılar vardır. Yığında, kuyrukta, dizide alıştığımız doğrusal yapıdan farklı olarak graf doğrusal olmayan bir yapıdır.

2.3 Sosyal ağlar ve diğer örnekler

Sosyal ağlarda düğümler kişilerdir. Kenarlar ise kişiler arasındaki ilişkilerdir: iş arkadaşı olmak, arkadaş olmak, aynı aileden olmak gibi. Facebook'u böyle bir ağ yapısında saklayabilirsiniz.

Hoca slayttaki uygulama tablosundan şu örnekleri de verdi:

- **İletişim (communication):** Düğümler telefonlar ya da bilgisayarlar, bağlantılar aralarındaki fiziksel bağlantılar ve kablolar.
- **Ulaşım (transportation):** Düğümler caddeler ve kavşaklar, bağlantılar aralarındaki yollar.
- **Devreler:** Düğümler kapılar, register'lar ve işlemciler, kenarlar (edge) bunları birbirine bağlayan teller.

Graf, pek çok gerçek dünya uygulamasında en çok ihtiyaç duyulan veri yapılarından biridir.

3. Graf Terminolojisi

Hoca terimleri anlatmanın biraz sıkıcı olduğunu, ama yeri geldiğinde hepsini kullanacağımız için bilmemiz gerektiğini söyledi.

3.1 Düğüm, kenar ve komşuluk

Grafta düğümler (vertex ya da node) ve kenarlar (edge) vardır. Örneğin bir A düğümü ve bir B düğümü ile aralarındaki bağlantı. Düğümlerin ve kenarların neyi göstereceğine uygulamaya göre biz karar veririz. Sosyal ağda düğümler kişileri, kenarlar kişiler arasındaki ilişkileri gösterebilir.

Bir grafta u ve v düğümleri varsa, bu iki düğümü birbirine bağlayan kenar (u, v) diye gösterilir. İki düğüm arasında bir kenar varsa bu iki düğüm komşudur (adjacent), yani birbirine bağlıdır.

3.2 Derece

Bir düğümün derecesi (degree), o düğüme bağlı olan kenar sayısıdır. Hoca slayttaki örnek grafta şunları gösterdi:

- A yalnızca D'ye bağlıdır, derecesi 1'dir.
- D; A, B, C ve E'ye bağlıdır, derecesi 4'tür.

- J; G ve E'ye bağlıdır, derecesi 2'dir.
- H'nin üç bağlantısı vardır, derecesi 3'tür.

3.3 Yol ve döngü

Yol (path), bir düğümden başka bir düğüme kenarlar üzerinden giden yoldur. "A'dan D'ye yol" dediğimizde A'dan D'ye giden yolu göstermiş oluruz. Yolun uzunluğu (length) üzerinden geçilen kenar sayısıdır. Hoca slayttaki örnekte birinci, ikinci, üçüncü ve dördüncü kenardan geçen, uzunluğu 4 olan bir yol gösterdi.

Döngü (cycle), başladığımız düğüme geri döndüğümüz yoldur. Slayttaki örnekte bir düğümden başlanıp beş kenardan geçilerek aynı düğüme dönülüyordu. Bu, uzunluğu 5 olan bir döngüdür.

3.4 Bağlı bileşen

Grafta birbirine bağlı olan düğümler bir bileşen (component) oluşturur. Hocanın gösterdiği örnek uzayda üç ayrı parça vardı, yani üç bağlı bileşen (connected component). Hoca bunu dünya üzerindeki kıtalara benzetti: birbirine değmeyen her toprak parçası bir bağlı bileşen gibi düşünülebilir.

3.5 Yönsüz ve yönlü graf

Gösterilen graflar yönsüzdü (undirected). A, B, C düğümlerinden oluşan bir grafta hiçbir yön bilgisi yoksa A'dan C'ye de gidebilirsiniz, C'den A'ya da. Arkadaşlık ilişkilerini gösterirken yönsüz graf kullanılabilir: Elif Ayşe ile arkadaşsa, Ayşe de Elif ile arkadaştır.

Karayolu haritasında ise yönlü graf vardır. Kadıköy'den Beşiktaş'a giderken kullandığınız yollar, Beşiktaş'tan Kadıköy'e dönerken kullandığınız yollarla bire bir aynı olmayabilir. Gidiş ve geliş şeritlerine, tek yönlü yollara bağlı olarak dönüşte aynı yolu kullanamayabilirsiniz. Yönlü graflar 9. bölümde ayrıntılı anlatılmıştır.

3.6 Ağaç ve yayılan ağaç

Graf yapısı ağaç yapısına da benzer. Ağaçta tek yönlü bir bilgi akışına ve bir hiyerarşiye alışkınız. Bir ağaçta ilerlerken bir düğümü sonradan başka bir düğüme de bağlarsanız artık ağaç olmaz, çünkü bir döngü oluşur. Hocanın tanımıyla: döngü oluşmuyorsa ve düğümler birbirine bağlıysa buna ağaç (tree) denir. Dolayısıyla her ağaç aynı zamanda bir graftır, ama döngü içeren bir graf ağaç değildir.

Ağacın özel bir çeşidi yayılan ağaçtır (spanning tree). Hoca bunu Ayrık Matematik dersinden hatırlayacağımızı söyledi. Graftaki bütün düğümlerin bağlı olduğu, ama her düğümden her düğüme bağlantı olması gerekmeyen ve döngü oluşturmayan ağaca yayılan ağaç denir. Slayttaki örnekte bütün düğümler bağlı ve döngü yoktu. Hoca, oraya bir kenar daha eklenirse döngü oluşacağını ve yapının artık ağaç olmayacağını gösterdi.

> **Ders sonrası not:** Hoca bir yerde "eğer bir grafta döngü yoksa buna ağaç deniyor" dedi, hemen ardından da "döngü oluşmuyorsa ve düğümler birbirine bağlıysa" diye tam tanımı verdi. Doğru tanım ikincisidir: ağaç bağlı ve döngüsüz bir graftır. Döngüsüz ama bağlı olmayan grafa orman (forest) denir.

3.7 Klik

Tüm düğümlerin birbirine bağlı olduğu yapıya klik (clique) denir. Örneğin A, B, C diye üç düğüm varsa ve hepsi birbirine bağlıysa bu bir kliktir. Bir öğrenci kliği kaçırdığını söyleyip tekrar sordu. Hoca slaytı büyüterek açıkladı: bütün düğümlerin birbirine bağlı olduğu alt grafa klik denir. Bu alt graf, grafın bir parçası olabileceği gibi grafın kendisi de olabilir.

3.8 Bağlı ve bağlı olmayan graf

Grafı oluşturan bütün düğümler birbirine bağlıysa, yani boşta kalan düğüm yoksa, buna bağlı graf (connected graph) denir. Hocanın gösterdiği ikinci örnekte ise aynı uzayda dört ayrı parça

vardı. Bu parçalar birbirine bağlı olmadığı için bu bir bağlı olmayan graftır (disconnected graph).

Hoca iki örnek verdi. Birincisi yine dünya üzerindeki kıtalardır. İkincisi bir üniversitedir: yalnızca fakülteleri gösteren bir graf düşünün. Her fakülte kendi içinde bağlı bir graftır, ama fakülteler arasında bağlantı yoktur. Böyle bir üniversite grafı bağlı olmayan bir graftır.

3.9 İki parçalı graf

Düğümler iki kümeye ayrılıyorsa ve kenarlar yalnızca farklı kümelerdeki düğümler arasındaysa buna iki parçalı graf (bipartite graph) denir. Hocanın örnekleri:

- Bir tarafta öğrenciler, diğer tarafta dersler. Öğrencilerin aldıkları dersleri göstermek istiyorsunuz. Öğrencilerle dersler arasında ilişki vardır, ama öğrencilerle öğrenciler ya da derslerle dersler arasında ilişki yoktur.
- Netflix'te bir tarafta üyeler, diğer tarafta filmler ve diziler. Kenarlar üyelerin seçtikleri filmleri gösterir.

Kümeler U ve V ise, iki parçalı olmanın şartı U'nun ya da V'nin kendi içinde hiç bağlantı olmamasıdır.

3.10 Ağırlıklı graf

Şimdiye kadar yalnızca bağlantının var olup olmadığından söz ettik. Ama bağlantılar genellikle bir değere sahiptir. Kadıköy ile Altunizade arasındaki mesafe 8 km olabilir. Ayşe ile Elif arasındaki bağlantı ilkokul arkadaşı olmaları olabilir. Kenarların sayısal bir değeri varsa buna ağırlıklı graf (edge-weighted graph) denir. Slayttaki örnekte 0. düğümle 1. düğüm arasındaki bağlantının değeri 3, diğer iki bağlantının değerleri 7 ve 4'tü.

4. Sembol Tablosu: Düğümleri Sayılarla Temsil Etmek

Gerçek dünyada düğümlere isim veririz: burası Elif, burası Ayşe, burası Ahmet ve aralarında bağlantılar var deriz. Ama bilgisayar ortamında, kodlamada kolaylık olsun diye düğümleri A, B, C, D ile değil 0, 1, 2, 3, … gibi sayılarla gösteririz.

O zaman isimleri saklamak için bir sembol tablosu (symbol table) kurmak mantıklıdır. Örneğin bir string diziniz olur ve indisler isimleri gösterir:

İndis	İsim
0	Ali
1	Ayşe
2	Banu
3	Bernay

Algoritmada hep 0, 1, 2, 3 ile işlem yaparsınız. Ama arayüzde kullanıcıya gösterirken 0 olan yerde "Ali", 2 olan yerde "Banu" gösterirsiniz. Yani gerçek dünyada nasıl kullanılıyorsa öyle gösterirsiniz.

> **Hocanın vurgusu:** Mantıklı olan budur. Yoksa kod yazarken, kimin kime bağlı olduğuyla çalışırken çok sıkıntı yaşarsınız.

5. Grafın Saklanması: Komşuluk Matrisi

Grafın mantığını anladık: bir düğüm var ve başka bir düğüme bağlı. Bu bağlantının bir değeri olabilir, hoca örnek olarak 90 dedi. Yani bir kenar, bir u düğümünü bir v düğümüne bağlar. Elimizde iki bilgi ve aralarındaki bağlantının değeri ya da bağlantının olup olmadığı varsa, yapılabilecek ilk şey bilgileri bir komşuluk matrisinde (adjacency matrix) saklamaktır.

Slayttaki örnek grafta 13 düğüm vardı, yani düğümler 0'dan 12'ye kadar numaralıydı. Graf 13×13'lük bir matriste saklanır ve her düğümün diğer düğümlerle bağlantısı gösterilir. Örneğin 0. düğümün satırına bakınca 1, 2, 5 ve 6'ya bağlı olduğu görülür. 0. sütuna bakınca da

aynı bilgi görülür: 0. düğüme bağlı olanlar 1, 2, 5 ve 6'dır. Yani bağlantı iki yönden de görünür. Bu yüzden yönsüz bir grafın komşuluk matrisi köşegene (diagonal) göre simetriktir.

Komşuluk matrisi iki türlü kullanılabilir:

- **Ağırlıksız graf:** Graf yalnızca bir düğümün başka bir düğüme bağlı olup olmadığını gösteriyorsa, bağlıysa 1, bağlı değilse 0 yazılır. Örneğin 0. düğüm 2. düğüme bağlıysa ilgili gözde 1 vardır.
- **Ağırlıklı graf:** Matris düğümler arasındaki bağlantıların ağırlıklarını içerir. Slayttaki örnekte 0 ile 1 arasındaki bağlantının değeri 2'dir: satırdan da sütundan da bakınca 2 görülür. 0 ile 2 arasındaki bağlantının değeri 3'tür.

5.1 Küçük bir örnek

Aradan sonra hoca bu gösterimleri tahtada küçük bir örnekle tekrarladı. Düğümler 1, 2, 3; kenarlar 1-2 ve 1-3. Hoca dilden bağımsız anlattığı için indisleri 1'den başlattı ve C'de normalde 0'dan başlanacağını hatırlattı.

	1	2	3
1	0	1	1
2	1	0	0
3	1	0	0

1 ile 2, 1 ile 3, 2 ile 1 ve 3 ile 1 arasında bağlantı vardır. Diğer gözlere 0 yazılır. Matrisin adı A ise $A[1][1] = 0$'dır. $A[3][2]$ gösteriminde ilk indis satırı, ikinci indis sütunu gösterir: 3. satır, 2. sütun, değeri 0'dır.

6. Yoğun ve Seyrek Graflar

İki tür graf vardır: yoğun graf (dense graph) ve seyrek graf (sparse graph).

Yoğun grafta düğümler arasındaki bağlantılar yoğundur. Bu, her düğümün mutlaka diğer her düğüme bağlı olması gerektiği anlamına gelmez. Hoca Google Maps grafını yoğun grafa örnek verdi, çünkü düğümler arasında epeyce bağlantı vardır. Bunun her zaman böyle olmayabileceğini de ekledi.

Seyrek grafta düğümler arasındaki bağlantılar azdır. Hocanın iki örneği:

- **Facebook:** Diyelim ki bir milyar kullanıcı var, ama bir kişinin en fazla 500 arkadaşı olabiliyor. Bunu bir milyarlık bir matrisle göstermeye çalışırsanız her satırda bir milyar göz olur, ama bu gözlerin en fazla 500'ü bir bağlantıyı gösterir.
- **Netflix:** Bir milyar üye ve 10.000 film ya da dizi olduğunu düşünelim. Üyelerden biri, diyelim Elif, bunların topu topu 500 tanesini izlemiş olsun. Elif'in satırında 10.000 gözden yalnızca 500'ünün değeri 1'dir. Bir kişi izlese izlese 500, taş çatlasa 1000 tane izler. Çoğunluk 10-20 tane izler. Bu graf aynı zamanda iki parçalı bir graftır: bir tarafta kişiler, diğer tarafta filmler vardır.

  Hocanın vurgusu: Çoğunluğunun boş olduğunu bildiğiniz bir grafı matriste saklamak, yerin verimli kullanılması açısından kötü bir tercihtir. Seyrek graflar için iyi çözüm, onları komşuluk listesinde (adjacency list) saklamaktır.

  Ders sonrası not: Hoca Facebook'ta bir kişinin en fazla 500 arkadaşı olabileceğini söyledi. Facebook'un arkadaş sınırı büyük olasılıkla 5000'dir. Örneğin vardığı sonuç değişmez: satır başına dolu göz sayısı bir milyarın yanında yine çok küçüktür.

7. Komşuluk Listesi ve C ile Gerçeklenmesi

7.1 Fikir

Komşuluk listesinde her düğüm için, o düğüme bağlı olan düğümlerin bilgisi bir bağlı liste yapısında saklanır. Neden bağlı liste? Çünkü her düğüm farklı sayıda düğüme bağlıdır. Slayttaki

grafta 0. düğüm dört düğüme bağlıyken 1. düğüm ve 2. düğüm yalnızca birer düğüme bağlıdır. Matris kullanmayacaksak, her satırın farklı uzunlukta eleman içerebildiği bir yapı gerekir. Bu yüzden her düğümün komşularını bir bağlı listeyle göstermek doğrudur.

Slayttaki listeden okunanlar (slayt komşuları sırasız göstermişti):

```
0 -> 5 -> 2 -> 1 -> 6
1 -> 0
3 -> 5 -> 4
5 -> 0 -> 4 -> 3
```

Komşuluk listesi yalnızca var olan bağlantıları içerdiği için grafın durumunu çok daha az yerle gösterir.

Tahtadaki küçük örnek için (düğümler 1, 2, 3; kenarlar 1-2 ve 1-3) liste şöyledir:

```
1 -> 2 -> 3 -> NULL
2 -> 1 -> NULL
3 -> 1 -> NULL
```

Soldaki 1, 2, 3 değerleri listelerin baş (head) göstergeleridir. Hoca, bunun ayrı kutularla da çizilebileceğini, kitaplarda genelde bu şekilde gösterildiğini söyledi.

7.2 Veri yapıları

Hoca komşuluk listesinin C'de nasıl yazıldığını adım adım gösterdi.

Önce bir düğüm yapısı tanımlanır. Listedeki her düğümde iki bilgi vardır: düğüm numarası (`vertex`) ve listedeki bir sonraki düğümün adresi (`next`). `next`, `struct node` tipinde bir göstergedir. Örneğin 1'in listesinde 2 vardır ve onun `next`'i bir sonraki elemanı gösterir.

Sonra graf yapısı tanımlanır. Grafta düğüm sayısı (`numVertices`) tutulur ki işlemlerde kullanılabilsin. Bir de liste başlarını tutan bir dizi vardır. Bu dizinin her elemanı bir liste başının adresini, yani bir göstergeyi sakladığı için dizi bir gösterge dizisidir. Bu yüzden tipi `struct node **` (göstergeye gösterge) olur.

7.3 Düğüm oluşturma, graf oluşturma ve kenar ekleme

`createNode(v)`: Yeni düğüm için `struct node` boyutunda bellek ayrılır (`malloc`). `vertex` alanına `v` yazılır. Düğüm daha yeni oluşturulduğu için arkasında kimse yoktur, `next` alanına `NULL` atanır. Birazdan bir listeye bağlanacaktır.

`createGraph(vertices)`: Önce graf için yer ayrılır. `numVertices` alanına grafın kaç düğümden oluştuğu yazılır. Sonra düğüm sayısı kadar gösterge için yer açılır ve dizinin her elemanı `NULL` ile ilklendirilir. Başlangıçta, örneğin 3 düğümlü bir grafta, 0, 1 ve 2'nin listelerinin hepsi `NULL`'ı gösterir. Yeni kenarlar geldikçe ilgili satıra eklenecektir.

`addEdge(graph, u, v)`: Yeni bir kenar gelmesi, u ile v arasında bir bağlantı olması demektir. Hoca u'yu ilk düğüm, v'yi hedef (destination) düğüm olarak adlandırdı. Önce v için yeni bir düğüm oluşturulur. Bu düğümün `next`'i u'nun listesinin eski başı yapılır, sonra u'nun liste başı yeni düğümü gösterir. Graf yönsüz olduğu için aynı işlem ters yönde de yapılır: u için yeni bir düğüm oluşturulup v'nin listesinin başına eklenir.

Derste yazılan fonksiyonlar ve bunları denemek için bir `main` aşağıdadır. `gcc -Wall -Wextra` ile uyarısız derlendi ve çalıştırıldı.

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int vertex;
    struct node *next;
};

struct Graph {
    int numVertices;
```

```c
    struct node **adjLists;   /* her düğüm için liste başı göstergesi */
};

struct node *createNode(int v) {
    struct node *newNode = malloc(sizeof(struct node));
    newNode->vertex = v;
    newNode->next = NULL;
    return newNode;
}

struct Graph *createGraph(int vertices) {
    int i;
    struct Graph *graph = malloc(sizeof(struct Graph));
    graph->numVertices = vertices;
    graph->adjLists = malloc(vertices * sizeof(struct node *));
    for (i = 0; i < vertices; i++)
        graph->adjLists[i] = NULL;          /* başlangıçta tüm listeler boş */
    return graph;
}

void addEdge(struct Graph *graph, int u, int v) {
    struct node *newNode = createNode(v);   /* u'nun listesinin başına v */
    newNode->next = graph->adjLists[u];
    graph->adjLists[u] = newNode;

    newNode = createNode(u);                /* yönsüz graf: v'nin listesinin başına u */
    newNode->next = graph->adjLists[v];
    graph->adjLists[v] = newNode;
}

/* Derste yazılmadı: listeleri ekrana basmak ve belleği bırakmak için. */
void printGraph(struct Graph *graph) {
    int i;
    for (i = 0; i < graph->numVertices; i++) {
        struct node *p = graph->adjLists[i];
        printf("%d:", i);
        while (p != NULL) {
            printf(" -> %d", p->vertex);
            p = p->next;
        }
        printf(" -> NULL\n");
    }
}

void freeGraph(struct Graph *graph) {
    int i;
    for (i = 0; i < graph->numVertices; i++) {
        struct node *p = graph->adjLists[i];
        while (p != NULL) {
            struct node *q = p->next;
            free(p);
            p = q;
        }
    }
    free(graph->adjLists);
    free(graph);
}

int main(void) {
    struct Graph *graph = createGraph(3);
```

```
    addEdge(graph, 0, 2);
    printGraph(graph);
    printf("--\n");
    addEdge(graph, 0, 1);
    printGraph(graph);
    freeGraph(graph);
    return 0;
}
```

Çıktı:

```
0: -> 2 -> NULL
1: -> NULL
2: -> 0 -> NULL
--
0: -> 1 -> 2 -> NULL
1: -> 0 -> NULL
2: -> 0 -> NULL
```

> **Ek bilgi (derste anlatılmadı):** `printGraph`, `freeGraph` ve `main` derste yazılmadı; yalnızca derste yazılan `createNode`, `createGraph` ve `addEdge` fonksiyonlarının hocanın izlediği adımlarla aynı sonucu verdiğini göstermek için eklendi. `malloc` ile alınan her düğüm iş bitince `free` ile bırakılmalıdır.

7.4 Hocanın izleme örneği: `addEdge(graph, 0, 2)`

Hoca, 0, 1, 2 düğümlü bir grafta `addEdge(graph, 0, 2)` çağrısının belleği nasıl değiştirdiğini adres numaralarıyla izledi. Başlangıçta 0'ın ve 2'nin listeleri `NULL`'dır.

1. `createNode(2)` çağrılır. Değeri 2 olan düğüm için 2500 numaralı adreste yer ayrıldığını varsayalım. `next`'i `NULL`'dır.
2. Bu düğümün `next`'i, 0'ın listesinin eski başı yapılır. Liste boş olduğu için bu `NULL`'dır.
3. `adjLists[0]` artık 2500'ü gösterir. 0, 2'ye bağlanmış olur.
4. `createNode(0)` çağrılır. Değeri 0 olan düğüm için 3000 numaralı adreste yer ayrılır.
5. Bu düğümün `next`'i, 2'nin listesinin eski başı yapılır. Bu da `NULL`'dır.
6. `adjLists[2]` artık 3000'i gösterir.

Sonuç: 0 → 2500 (içinde 2) → `NULL`; 2 → 3000 (içinde 0) → `NULL`.

Hoca neden her seferinde başa eklediğimizi bağlı listelerden hatırlattı: yeni eleman listenin sonuna değil başına eklenir ki boş yere listenin sonuna kadar gidip vakit harcamayalım. Başlangıçta listeler boş olduğu için ilk eklenen düğümün `next`'i her seferinde `NULL` olur.

7.5 İkinci kenar: 0 ile 1'in bağlanması

Hoca sonra 0'ın listesine 1'in nasıl ekleneceğini gösterdi. 1 için 4000 numaralı adreste yeni bir düğüm ayrılır. Bu düğümün `next`'i 0'ın listesinin o anki başı, yani 2500 yapılır. Sonra `adjLists[0]` 4000'i gösterir. Böylece 0'ın listesi 0 → 1 → 2 → `NULL` olur. Yeni gelen her komşu listenin başına eklenir, eskiler arkaya itilir.

> **Ders sonrası not:** Hoca, 0 için 3000 numaralı adreste zaten yer tutulduğunu, bu yüzden 0 için bir daha yer tutmayacağını ve 1'in listesinin 3000'deki düğümü göstereceğini söyledi. `addEdge` kodu böyle çalışmaz: her çağrıda `createNode(u)` ile yeni bir düğüm oluşturur. 1'in listesindeki 0, yeni bir adreste duran ayrı bir düğümdür. Aynı düğüm iki listede paylaşılsaydı, 2'nin listesinin devamı ile 1'in listesinin devamı aynı bellek olurdu ve bir listeye yapılan ekleme diğerini de bozardı. Yukarıdaki programın çıktısında 1'in ve 2'nin listeleri ayrı ayrı `1: -> 0` ve `2: -> 0` olarak görünür.

8. Komşuluk Matrisi ile Komşuluk Listesinin Karşılaştırılması

8.1 Bellek

Hoca 0, 1, 2 düğümlü, iki kenarlı (0-1 ve 0-2) bir grafla karşılaştırdı. Komşuluk listesi şöyledir: 0, 1'e ve 2'ye bağlıdır; 1, 0'a bağlıdır; 2, 0'a bağlıdır. Komşuluk matrisi ise 3×3'lük bir matristir ve bütün olası bağlantıları gösterir.

- **Komşuluk matrisi:** Kaç düğüm varsa o kadar satır ve sütun gerekir. 3 düğüm için $3\times3 = 9$ göz, genel olarak V^2 yer gerekir.
- **Komşuluk listesi:** Düğüm sayısı artı kenar sayısı kadar, yani $V + E$ kadar yer gerekir. Yönsüz grafta her kenar iki kez görünür: 0'ın 1'e bağlı olması bir bağlantı, 1'in 0'a bağlı olması bir bağlantı daha. 0-2 kenarı için de aynısı geçerlidir. Bu örnekte 3 liste başı ve 4 liste düğümü vardır (ara adım eklendi). Hiç kimseye bağlı olmayan bir düğüm olsaydı, onun için de `NULL` gösteren bir liste başı tutmak gerekirdi. Bu yüzden yer $V + E$ kadardır.

  Ders sonrası not: Yönsüz grafta liste düğümü sayısı tam olarak $2E$'dir, liste başlarıyla birlikte toplam $V + 2E$ eleman tutulur. Sabit çarpan büyük-O gösterimini değiştirmediği için bellek yine $O(V + E)$'dir.

8.2 "u ile v arasında kenar var mı?"

Komşuluk matrisinde, örneğin "0. düğümden 1. düğüme bağlantı var mı?" sorusu için yalnızca $A[0][1]$'e bakılır: 1 mi 0 mı? Bu işin karmaşıklığı $O(1)$'dir.

Komşuluk listesinde ise doğrudan erişim (direct access) yoktur, çünkü yapı bir listedir. Hocanın örneği: 0. düğüm 1'e, 2'ye, 3'e ve 4'e bağlı olsun. "0, 4'e bağlı mı?" sorusuna cevap vermek için en kötü durumda listenin sonuna kadar gitmek gerekir. Hoca bunun en kötü durumda kenar sayısı kadar iş olduğunu söyledi. Bu, komşuluk listesinin bir dezavantajıdır.

8.3 "v kimlere doğrudan bağlı?"

Komşuluk matrisinde, örneğin 2. düğümün kimlere bağlı olduğunu öğrenmek için 2. satırdaki değeri 1 olan gözleri bulmak gerekir. Bunun için satırdaki bütün gözler tek tek kontrol edilir: 2'den 0'a bağlantı var mı, 2'den 1'e var mı, 2'den 3'e var mı... Satır uzunluğu düğüm sayısı kadar olduğu için karmaşıklık $O(V)$'dir.

Komşuluk listesinde ise o satırda yalnızca gerçekten bağlı olan düğümler vardır. Grafta bin düğüm olabilir, ama düğüm iki üç düğüme bağlıysa listede yalnızca onlar görünür.

> **Ders sonrası not:** Hoca bu işlemin komşuluk listesinde $O(1)$ olduğunu söyledi. Doğrusu $O(\deg(v))$'dir: listede kaç komşu varsa o kadar eleman okunur. Bu, matristeki $O(V)$'den genellikle çok daha azdır, ama sabit değildir. Benzer şekilde 8.2'deki kenar sorgusu listede en kötü durumda u'nun derecesi kadar sürer, $O(\deg(u))$; bu da en fazla $V - 1$'dir.

İşlem	Komşuluk matrisi	Komşuluk listesi
Bellek	$O(V^2)$	$O(V + E)$
u ile v arasında kenar var mı?	$O(1)$	$O(\deg(u))$
v'nin bütün komşularını listele	$O(V)$	$O(\deg(v))$

8.4 Öğrenci sorusu: dolaylı bağlantı

Soru: Bir öğrenci, 0'dan 1'e, 1'den de 2'ye gidilebiliyorsa 0'ın 2'ye bağlı olup olmadığını anlamak için gezmek gerekeceğini, burada karmaşıklığın farklı olacağını sordu.

Cevap: Hoca sorunun farklı olduğunu söyledi. Buradaki soru "0 hangi düğümlere doğrudan bağlı?" sorusudur. Matriste bunun için 0. satırdaki n gözün hepsine tek tek bakmak gerekir: bire bağlı mı, ikiye bağlı mı, üçe bağlı mı. Listede ise cevap doğrudan listededir: 0, 1'e ve 2'ye bağlıdır. "0'dan 2'ye gidebilir miyiz?", "0'dan 3'e yol var mı?" gibi sorular ise başka bir konudur ve graf üzerinde gezinme (graph traversal) algoritmalarıyla cevaplanır. Hoca bunun iyi bir soru olduğunu söyledi.

Hoca, graf gezinme ve Dijkstra gibi konuların bu dönem uzaktan eğitim nedeniyle büyük ihtimalle yetişmeyeceğini, önümüzdeki sene verilme ihtimalinin daha yüksek olduğunu söyledi. Bu algoritmalar "Kadıköy'den kalktım, Beşiktaş'a nasıl giderim?" ya da "İstanbul'dan Londra'ya karayoluyla gitmek mümkün mü?" gibi sorulara cevap verir.

9. Yönlü Graflar

Bağlantılar arasında yönün bir anlamı varsa buna yönlü graf (directed graph, digraph) denir. Kadıköy-Beşiktaş örneğinde olduğu gibi, yolların yönüne bağlı olarak giderken geçtiğimiz yerlerden dönerken bire bir geçemeyiz. Slayttaki yol haritasında da bir yerden bir yere gitmek mümkündü ama geri gelmek mümkün değildi.

Hocanın diğer örnekleri:

- **Web sayfaları ve köprüler (hyperlink):** Bir gazetede yazarlar sayfasından A yazarının sayfasına, oradan A yazarının 20 Nisan tarihli yazısına gidersiniz. Tarayıcıda geri gelebilirsiniz, ama sayfaları bağlayan köprüler mantıken yönlüdür. Çift yönlü köprüler de olabilir.
- **Salgın takibi:** Düğümler kişilerdir. A kişisi hastalığı B'ye bulaştırır, B C'ye, A D'ye, E F'ye bulaştırır. Hoca, COVID döneminde bu grafların çok çizildiğini söyledi.

9.1 Matris ve listedeki fark

Yönsüz grafta 0-1 ve 1-2 kenarları olsun. Matriste hem $A[0][1]$ hem $A[1][0]$, hem $A[1][2]$ hem $A[2][1]$ doludur, çünkü bağlantı iki yönlüdür. Listede de 0 → 1; 1 → 2, 0; 2 → 1 yazılır, herkes birbirine bağlı görünür. Yönlü grafta ise bilgiler matriste ve listede yalnızca yönüyle gösterilir.

Slayttaki yönlü grafta 0. düğümün bağlı olduğu düğümler 1, 2, 5 ve 6'dır. Ama 5'in bağlı olduğu düğümler yalnızca 3 ve 4'tür. 5 artık 0'a bağlı görünmez, çünkü çift yönlü bir mantık yoktur. Komşuluk listesinde de 0 → 1, 2, 5, 6 ve 5 → 3, 4 yazılır.

Yönlü grafta $A[u][v] = A[v][u]$ olmak zorunda değildir. Eşit olabilir de, olmayabilir de; böyle bir kural yoktur. Dolayısıyla yönlü grafın komşuluk matrisi simetrik olmak zorunda değildir.

9.2 Giriş ve çıkış derecesi

Yönsüz grafta 0-1-2 zincirinde 0 numaralı düğümün derecesi 1, 1 numaralı düğümün derecesi 2'dir. Yönlü grafta ise 0'dan 1'e bağlantı olması 1'den 0'a bağlantı olduğu anlamına gelmez. Bu yüzden iki yeni kavram ortaya çıkar:

- **Giriş derecesi (in-degree):** Düğüme gelen kenar sayısı.
- **Çıkış derecesi (out-degree):** Düğümden çıkan kenar sayısı.

Slayttaki grafta 6 numaralı düğüme yalnızca bir kenar gelir, giriş derecesi 1'dir. Çıkan kenarlar üç tanedir, çıkış derecesi 3'tür.

Döngü kavramı da yönlü grafta değişir: kenarların yönüne uyarak başlangıç düğümüne dönülmesi gerekir. Slaytta 0 ile 2 arasında bir yol da görülüyordu. Hoca, yolun nasıl bulunduğunun graf gezinme algoritmalarında anlatılacağını söyledi.

10. Sembol Graf Örneği: Havaalanları

Hoca sembol graf (symbol graph) için havaalanlarından oluşan bir örnek gösterdi. Hocanın belirttiğine göre slayttaki örnek yönsüz bir graftı. JFK dendiğinde 0 sayısıyla, Houston dendiğinde 4 sayısıyla gösterileceğini bir yerde saklamanız gerekir. Ters yönde, 4 dendiğinde bunun Houston'a karşılık geldiğini bulmak için de ters bir indeks (inverted index) tutulabilir. Erişim kolaylığı açısından iki yön de saklanabilir.

0 JFK'yi gösterdiği için JFK'nin satırına bakınca 2'ye (ORD), 7'ye (Atlanta) ve 1'e bağlı olduğu görülür. Bilgisayarda saklarken 0'ın listesine 2, 7 ve 1 yazılır. Ama arayüzde kullanıcıya isimler gösterilir.

11. Minimum Yayılan Ağaç

Hoca, minimum yayılan ağacı (minimum spanning tree, MST) Ayrık Matematik dersinden duymuş olduğumuzu, ama orada nasıl oluşturulduğunun anlatılmadığını, bu derste nasıl tasarlanıp oluşturulduğunu göreceğimizi söyledi.

Önce tanımları hatırlattı. Bir graf yönsüz ve döngüsüzse (acyclic) bir ağaçtır. İçinde döngü olan bir graf ağaç değildir. Yönlü bir graf da ağaç sayılmaz. Graftaki bütün düğümleri içeren ağaca yayılan ağaç denir. Hoca tahtaya bir graf çizip bazı kenarlarını seçerek bir yayılan ağaç elde etti.

Kenarların ağırlıkları olan bir grafta, yayılan ağaçtaki kenar ağırlıklarının toplamı en küçükse bu ağaca minimum yayılan ağaç denir.

Hocanın kullanım örneği bir bilgisayar ağıdır. Ağdaki bütün bilgisayarların bağlı olmasını istiyorsunuz. Herkes herkese bağlı olmak zorunda değil, ama bütün bilgisayarlar aynı ağın içinde olmalı. Bunu da olabildiğince kısa bağlantılarla sağlamak istiyorsunuz. Minimum yayılan ağaç, bağlantıların toplamının en az olduğu durumdur. Tahtadaki örnekte seçilen kenarların ağırlıkları 3, 5, 8, 12 ve 17'ydi. Bu kenarlarla bağlantıların en kısa olduğu yayılan ağaç elde edilmiş oldu.

11.1 Örnek graf

Prim ve Kruskal algoritmaları aynı graf üzerinde gösterildi. Düğümler 0'dan 7'ye kadardır, kenarlar ve ağırlıklar şöyledir:

Kenar	Ağırlık	Kenar	Ağırlık
0-1	32	1-7	21
0-2	29	3-4	34
0-5	60	3-5	18
0-6	51	4-5	40
0-7	31	4-6	51
6-7	25	4-7	46

12. Prim Algoritması

Hoca Prim algoritmasının 1930 yılında bulunduğunu söyledi. Prim açgözlü (greedy) bir algoritmadır. Hoca açgözlü yaklaşımı şöyle tanımladı: yer bulma problemlerinde seçilen en iyi yerel çözümün, problemin tamamının çözümünde de en iyi sonucu vereceği varsayımına dayanır. Yani yerelde en iyi sonucu veren seçim globalde de en iyi sonucu verecektir. Ağaç her adımda en kısa bağlantı seçilerek büyütülür.

Slayttaki ifade: 0 düğümünden başla ve ağacı adım adım büyüt. Her adımda, ağaçta tam olarak bir ucu bulunan en ucuz kenarı ekle. Döngü oluşturan kenar eklenmez, çünkü yapının ağaç olarak kalması gerekir.

> **Ders sonrası not:** Algoritma büyük olasılıkla ilk kez 1930'da Jarník tarafından yayımlanmıştır; Prim aynı algoritmayı 1957'de yeniden bulmuştur. Hocanın verdiği 1930 tarihi bu ilk yayına karşılık gelir.
>
> **Ders sonrası not:** "Yerelde en iyi seçim globalde de en iyidir" genel bir kural değildir. Açgözlü seçim birçok problemde en iyi sonucu vermez. MST için ise açgözlü seçimin doğruluğu kanıtlanmıştır (kesme özelliği, cut property): bir düğüm kümesini geri kalan düğümlerden ayıran kenarlar arasındaki en hafif kenar bir MST'de yer alır.

12.1 Adım adım çözüm

Hoca 0 düğümünden başladı. Her adımda ağaçtaki düğümlerden çıkan aday kenarları topladı, döngü oluşturanları sildi ve en küçüğünü seçti.

1. **Ağaç: {0}.** 0'ın kenarları küçükten büyüğe: 0-2 (29), 0-7 (31), 0-1 (32), 0-6 (51), 0-5 (60). En kısa bağlantı 0-2'dir. **0-2 (29) eklenir.**
2. **Ağaç: {0, 2}.** 2'nin başka bağlantısı olmadığı için yine 0'ın kenarlarına bakılır: 31, 32, 51, 60. **0-7 (31) eklenir.**
3. **Ağaç: {0, 2, 7}.** 7'nin kenarları da adaylara katılır: 7-1 (21), 7-6 (25), 7-4 (46). Adayların hepsi bir bütün olarak değerlendirilir: 21, 25, 32, 46, 51, 60. **7-1 (21) eklenir.** Artık 0-1 alınamaz: 0, 7 ve 1 zaten ağaçtadır, 0-1 eklenirse 0-7-1 döngüsü oluşur. Hoca kenarı bağlayıp döngüyü gösterdi ve geri aldı. 1'in başka bağlantısı yoktur.
4. **Ağaç: {0, 1, 2, 7}.** Adaylar: 7-6 (25), 7-4 (46), 0-6 (51), 0-5 (60). **7-6 (25) eklenir.** 6'nın bağlantıları işin içine girer: 0 zaten ağaçta olduğu için 0-6 artık döngü oluşturur ve silinir. 6-4 (51) adaylara eklenir.
5. **Ağaç: {0, 1, 2, 6, 7}.** Adaylar: 7-4 (46), 4-6 (51), 0-5 (60). **7-4 (46) eklenir.** Şimdi 4 de 6 da ağaçtadır; 4-6 eklenirse döngü oluşur. Hoca bunu da tahtada gösterdi ve 4-6'yı sildi.
6. **Ağaç: {0, 1, 2, 4, 6, 7}.** Hoca bir an 0-5'i de silmek üzereydi, sonra düzeltti: 5 henüz ağaçta olmadığı için 0-5 hâlâ adaydır. 4'ün kenarları eklenir: 4-3 (34), 4-5 (40). Adaylar: 34, 40, 60. **4-3 (34) eklenir.**
7. **Ağaç: {0, 1, 2, 3, 4, 6, 7}.** 3'ün kenarı 3-5 (18) eklenir. Adaylar: 18, 40, 60. **3-5 (18) eklenir.** Bütün düğümler bağlanmıştır.

Adım	Eklenen kenar	Ağırlık
1	0-2	29
2	0-7	31
3	7-1	21
4	7-6	25
5	7-4	46
6	4-3	34
7	3-5	18

Toplam ağırlık (ara adım eklendi): $29 + 31 + 21 + 25 + 46 + 34 + 18 = 204$. Bu iz ve toplam python ile doğrulandı.

13. Kruskal Algoritması

Hoca, Prim için "başlangıç noktasına bağlı olarak her zaman doğru sonucu bulamayabilirsiniz, başlangıç noktası bir aşamada önemli" dedi. Kruskal algoritması minimum yayılan ağacı bulmaya başka bir şekilde yaklaşır: bağlantıları küçükten büyüğe sıralar ve en küçükten başlayarak sırayla alır. Ne de olsa istenen toplamın minimum olmasıdır.

Hoca iki algoritmanın farkını şöyle özetledi. Prim bir noktadan başlar ve açgözlü yaklaşımla yalnızca ağaca bağlı düğümlerin henüz kullanılmamış en küçük bağlantılarına bakar. Kruskal ise "bir bağlantıdan genişleyerek büyümek yerine global bakalım" der: eninde sonunda bütün düğümler ağaca girmek zorunda olduğuna göre bütün kenarlar ağırlıklarına göre sıralanır ve bu sırayla değerlendirilir. Kenar döngü oluşturmuyorsa alınır.

13.1 Adım adım çözüm

Sıralı kenar listesi: 3-5 (18), 1-7 (21), 6-7 (25), 0-2 (29), 0-7 (31), 0-1 (32), 3-4 (34), 4-5 (40), 4-7 (46), 0-6 (51), 4-6 (51), 0-5 (60).

Sıra	Kenar	Ağırlık	Karar
1	3-5	18	alınır
2	1-7	21	alınır
3	6-7	25	alınır
4	0-2	29	alınır
5	0-7	31	alınır
6	0-1	32	alınmaz: 0-7-1 ile döngü oluşur
7	3-4	34	alınır

Sıra	Kenar	Ağırlık	Karar
8	4-5	40	alınmaz: 4-3-5 ile döngü oluşur
9	4-7	46	alınır, 7 kenar oldu, bütün düğümler bağlandı

Hoca ilk anlatımda 0-1 kenarını da aldı. 4-5'in döngü oluşturduğunu gösterip 4-7 ile bitirdiğinde bir öğrenci araya girdi.

Soru: Orada da döngü yok mu? (Öğrenci "0-7" dedi; kastettiği, 0-7'den sonra alınan 0-1 kenarıydı.)

Cevap: Hoca öğrencinin haklı olduğunu söyledi, teşekkür etti ve 0-2'ye kadar geri dönüp yeniden ilerledi. 0-7 (31) alınır. 0-1 alınırsa 0-7-1 döngüsü oluşacağı için 0-1 alınmaz. Sonra 3-4 alınır, 4-5 döngü oluşturduğu için alınmaz, 4-7 alınır ve bütün düğümler bağlanır.

Hoca sonra Prim ile seçilen kenarları karşılaştırdı: 3-5 (18), 1-7 (21), 6-7 (25), 0-2 (29), 0-7 (31), 3-4 (34) ve 4-7 (46). İki algoritma aynı minimum yayılan ağacı oluşturdu, toplam ağırlık yine 204'tür.

Hoca bunun her zaman böyle olması gerekmediğini, iki algoritmanın da tam doğru sonucu bulamadığı durumlar olabileceğini söyledi. Prim algoritmasında başlangıç noktasına bağlı olarak minimum olmayan, daha kısası bulunan bir yayılan ağaç oluşturma ihtimali olduğunu ekledi.

> **Ders sonrası not:** Hoca Prim'in başlangıç noktasına bağlı olarak minimum olmayan bir yayılan ağaç bulabileceğini, Kruskal'ın da her zaman doğru sonucu bulamayabileceğini söyledi; doğrusu şudur: bağlı, ağırlıklı ve yönsüz bir grafta Prim hangi düğümden başlarsa başlasın bir minimum yayılan ağaç bulur, Kruskal da her zaman bir minimum yayılan ağaç bulur. Aynı ağırlıkta kenarlar varsa farklı başlangıçlar ya da farklı sıralamalar farklı ama toplamı eşit ağaçlar verebilir. Bu dersteki grafta Prim sekiz başlangıç düğümünün her birinden çalıştırıldı ve hepsinde toplam 204 bulundu. Grafın 162 yayılan ağacı tek tek sayıldığında toplamı 204 olan tek bir ağaç olduğu, bir sonraki en küçük toplamın 205 olduğu görüldü (python ile doğrulandı).

14. Döngü Kontrolü: Etiket Dizisi

Tahtada gözle bakarken "0-1'i alırsak döngü oluşur, o yüzden almamalıyız" dedik. Hoca, bunun program içinde nasıl anlaşılacağını gösterdi. Kruskal algoritmasının kodunu önümüzdeki hafta birlikte tasarlayacaklarını söyledi.

14.1 Döngü ne zaman oluşur?

0, 1 ve 7 düğümlerine bakalım. 0-7'yi aldıysam hem 0 hem 7 seçilmiştir. 7-1'i aldıysam 1 de seçilmiştir. Şimdi 0-1'e bakıyorum: hem 0 hem 1 daha önce alınmışsa bu kenar alınmamalıdır.

Hocanın ikinci örneği: düğümler 0, 1, 2, 3. 0-1 ve 1-2 alınmış, sıra 0-3'te. 0 alınmış ama 3 alınmamış, o zaman 0-3 alınabilir, döngü oluşmaz. Ardından 2-3'e bakılırsa hem 2 hem 3 alınmıştır, 2-3 alınmamalıdır.

Ama yalnızca "iki uç da daha önce alınmış mı?" diye bakmak yetmez. Hocanın üçüncü örneğinde 0'dan 7'ye kadar düğümleri olan bir grafta önce birkaç kenar alınmış ve iki ayrı parça oluşmuştu. Sıra 3-4'e geldiğinde hem 3 hem 4 daha önce alınmıştı. Hoca "3-4'ü almalı mıyım?" diye sordu. Cevap: alınmalıdır, çünkü 3 ile 4 aynı alt grafta değildir. 3-4 kenarı iki ayrı alt grafı birbirine bağlar, döngü oluşturmaz.

Karşılaştırma için hoca şu durumu gösterdi: 0-1, 1-2 ve 2-3 alınmışsa 0 ile 3 aynı alt graftadır, 0-3 alınmamalıdır. Demek ki iki şey kontrol edilir: düğümlerin alınıp alınmadığı ve aynı alt grafta olup olmadıkları.

14.2 Etiket dizisi

Bunu anlamak için her düğümün hangi alt grafa ait olduğu bir dizide, etiket (label) dizisinde tutulur. Başlangıçta hiçbir düğüm alınmamıştır, bütün etiketler 0'dır. Bir (u, v) kenarı değerlendirilirken dört durum vardır:

- **İkisi de 0:** İki düğüm de henüz hiçbir alt grafta değildir. İkisi de yeni bir etiketle etiketlenir, kenar alınır. Örneğin daha önce 1 ve 2 etiketleri kullanılmışsa yeni etiket 3 olur.
- **Biri 0, diğeri x:** 0 olan düğüme x etiketi verilir, kenar alınır. Örneğin bir uç 0, diğeri 5 ise 0 olan uç 5 ile etiketlenir.
- **Biri x, diğeri y ($x \neq y$):** İki düğüm iki ayrı alt graftadır. Kenar alınır ve iki alt graf birleşir: bütün y etiketleri x yapılır ya da bütün x etiketleri y yapılır, hangisi olduğunun önemi yoktur. Örneğin 3 ile 4 bağlanırsa artık 4 ile 5 de aynı etiketle etiketlenmelidir, çünkü hepsi tek bir grafın parçası olmuştur.
- **İkisi de x:** İki düğüm zaten aynı alt graftadır. Hoca sınıfa "ikisi de x ise ne yaparsınız?" diye sordu, bir öğrenci "bağlamıyoruz" dedi. Hoca "evet, çok güzel" dedi. Kenar alınmaz, alınırsa döngü oluşur.

14.3 Örnek graf üzerinde etiket izi

Hoca Kruskal'ı aynı graf üzerinde etiket dizisiyle, alt grafları farklı renklerle boyayarak yeniden işletti. Tablodaki sütunlar 0'dan 7'ye kadar düğümlerin etiketleridir.

Kenar	Durum	Karar	0	1	2	3	4	5	6	7
başlangıç			0	0	0	0	0	0	0	0
3-5 (18)	ikisi de 0	al, yeni etiket 1	0	0	0	1	0	1	0	0
1-7 (21)	ikisi de 0	al, yeni etiket 2	0	2	0	1	0	1	0	2
6-7 (25)	6: 0, 7: 2	al, 6 ← 2	0	2	0	1	0	1	2	2
0-2 (29)	ikisi de 0	al, yeni etiket 3	3	2	3	1	0	1	2	2
0-7 (31)	0: 3, 7: 2	al, bütün 3'ler 2 olur	2	2	2	1	0	1	2	2
0-1 (32)	ikisi de 2	alma	2	2	2	1	0	1	2	2
3-4 (34)	3: 1, 4: 0	al, 4 ← 1	2	2	2	1	1	1	2	2
4-5 (40)	ikisi de 1	alma	2	2	2	1	1	1	2	2
4-7 (46)	4: 1, 7: 2	al, bütün 2'ler 1 olur	1	1	1	1	1	1	1	1

0-7 adımında 0'ın bulunduğu alt grafın etiketi 3, 7'ninki 2'dir. İki ayrı alt graf oldukları için kenar alınır. Hoca bütün 3'leri 2 yaptı; böylece 0, 1, 2, 6, 7 aynı alt graf oldu. Döngü oluşmadı ama hepsi birbirine bağlandı. 4-7 adımında hoca bütün 1'lerin 2 ya da bütün 2'lerin 1 yapılabileceğini, bunun önemli olmadığını söyledi ve 2'leri 1 yaptı. 4-7 alındığında herkes birbirine bağlanmış oldu. Bu iz python ile aynı kurallarla işletilerek doğrulandı.

> **Ek bilgi (derste anlatılmadı):** İki alt graf birleşirken bütün etiketleri değiştirmek dizinin tamamını taramayı gerektirir, yani her birleştirme $O(V)$ sürer. Pratikte bu iş için ayrık küme birleşim-bul (union-find, disjoint set) yapısı kullanılır. Bu yapıda "aynı kümede mi?" sorusu ve birleştirme, yol sıkıştırma ve rütbeye göre birleştirme ile neredeyse sabit zamanda yapılır.

15. Kruskal Algoritmasının Taslağı

15.1 MST'de kaç kenar olur?

Hoca sınıfa sordu: n düğümlü bir grafın minimum yayılan ağacında kaç kenar olur? Bir öğrenci "$n-1$ mi?" dedi. Hoca "harika" dedi ve açıkladı: iki düğüm varsa ağaçta bir kenar olur. Üç düğüm varsa iki kenar olur. Her zaman n düğüm varsa $n-1$ kenar vardır, çünkü bir kenar daha eklendiği anda döngü oluşur.

15.2 Algoritma

Hoca algoritmayı sözde kodla (pseudocode) yazdı. Girdi $G = (V, E)$ grafıdır ve grafta n düğüm vardır.

```
Kruskal(G):
    kenarları ağırlıklarına göre küçükten büyüğe sırala
    T = boş küme
    T'de n - 1 kenar olana kadar:
        sıradaki en küçük kenarı (u, v) seç
        eğer (u, v) iki farklı bağlı bileşeni bağlıyorsa:
            (u, v)'yi T'ye ekle
```

Hocanın örneği: 1-2 ve 3-4 seçilmiş olsun. 2-3'e bakılıyor. 1-2 bir bağlı bileşen, 3-4 başka bir bağlı bileşendir. 2-3 bu ikisini birbirine bağladığı için alınabilir. Ardından 2-4'e bakılırsa: 2-3 alındıktan sonra 1, 2, 3, 4 artık aynı bağlı bileşendedir, dolayısıyla 2-4 alınmamalıdır.

16. Çalışma Süreleri

16.1 Prim

Her adımda ağaca bir düğüm eklendiği için $V-1$ iterasyon olur. Her iterasyonda yapılan iş en küçük bağlantıyı bulmaktır. En kötü durumda, örneğin 0'dan yola çıkarken kimin alınacağına karar vermek için 0'ın bağlı olduğu bütün düğümlerle mesafesine bakmak gerekir; V düğüm varsa bu $O(V)$'dir. Hoca ayrıca her iterasyonda, ağaca alınan düğüm v için kalan düğümlerin mesafe (w) bilgisinin güncelleneceğini söyledi. İç içe iki döngü olduğu için:

$$T_{\text{Prim}} = O(V) \cdot O(V) = O(V^2)$$

16.2 Kruskal

Kruskal'da kenarlar sıralanır ve sırayla bakılır. Karmaşıklık sıralamanın karmaşıklığıdır: E kenar için $O(E \log E)$. Hoca bunun $O(E \log V)$ olduğunu söyledi.

Hoca, kenar sayısına bağlı olarak Kruskal'ın logaritmik çalıştığı için sonuca çok daha hızlı ulaşacağını söyledi.

> **Ders sonrası not:** Basit bir grafta $E \leq V^2$ olduğu için $\log E \leq 2 \log V$'dir; bu yüzden $O(E \log E) = O(E \log V)$ yazılabilir. "Kruskal daha hızlıdır" sonucu ise grafa bağlıdır. Yoğun bir grafta E, V^2 mertebesindedir; o zaman Kruskal $O(V^2 \log V)$ sürer ve matrisli Prim'in $O(V^2)$'si daha iyidir. Seyrek graflarda ise Kruskal daha avantajlıdır. Etiket dizisi kullanılırsa her birleştirmenin $O(V)$ sürdüğü de hesaba katılmalıdır.
>
> **Ek bilgi (derste anlatılmadı):** Prim algoritması komşuluk listesi ve bir öncelik kuyruğu (ikili heap) ile gerçeklenirse çalışma süresi $O(E \log V)$ olur. Seyrek graflarda bu, matrisli $O(V^2)$ sürümden daha iyidir.

Hoca dersi burada bitirdi. Kruskal algoritmasının kodunun nasıl çalıştığını önümüzdeki hafta anlatacağını söyledi.

Örnek Sorular

Bu derse ait hazır kare ve video olmadığı için ekrandan soru alınamadı. Aşağıdaki sorular derste sözlü olarak sorulan sorulardır.

Soru 1 (derste sözlü, 00:29:00)

Klik (clique) nedir?

> **Çözüm:** Bütün düğümlerin birbirine bağlı olduğu alt graftır. Bu alt graf grafın bir parçası olabileceği gibi grafın kendisi de olabilir. Örneğin A, B, C düğümlerinin üçü de birbirine bağlıysa {A, B, C} bir kliktir.

Soru 2 (derste sözlü, 01:26:11)

0, 1, 2 düğümlü ve 0-1, 0-2 kenarlı yönsüz bir graf komşuluk matrisinde ve komşuluk listesinde ne kadar yer tutar?

> **Çözüm:** Komşuluk matrisi $V^2 = 3 \times 3 = 9$ göz tutar. Komşuluk listesi $V + E$ mertebesinde yer tutar: 3 liste başı ve her kenar iki listede göründüğü için 4 liste düğümü vardır (0 → 1, 2; 1 → 0; 2 → 0). Hiçbir yere bağlı olmayan bir düğüm de olsaydı onun için de bir liste başı tutulurdu.

Soru 3 (derste sözlü, 01:30:23)

Bir öğrenci sordu: 0'dan 1'e, 1'den 2'ye gidilebiliyorsa, 0'ın 2'ye ulaşıp ulaşamadığını anlamak için grafı gezmek gerekmez mi, karmaşıklık bu durumda nedir?

> **Çözüm:** Hoca, 8. bölümdeki sorunun "0 hangi düğümlere doğrudan bağlı?" olduğunu söyledi: matriste 0. satırın bütün V gözüne bakılır, listede yalnız 0'ın listesi okunur. "0'dan 2'ye yol var mı?" sorusu ayrı bir konudur ve graf gezinme algoritmalarıyla cevaplanır. Bu konu bu derste işlenmedi.

Soru 4 (derste sözlü, 02:20:31)

Kruskal algoritması örnek grafta işletilirken 0-2 (29) ve 0-7 (31) alındıktan sonra 0-1 (32) kenarı alınmalı mı?

> **Çözüm:** Alınmamalıdır. Daha önce 1-7 (21) alınmıştı. 0-7 de alındığına göre 0, 7 ve 1 aynı bileşendedir; 0-1 eklenirse 0-7-1 döngüsü oluşur. Bir öğrenci bunu fark etti, hoca düzeltti. Etiket dizisiyle bakıldığında da 0'ın ve 1'in etiketi aynıdır (ikisi de 2).

Soru 5 (derste sözlü, 02:25:08)

0, 1, 2, 3 düğümlerinden oluşan bir grafta 0-1 ve 1-2 alınmış olsun. 0-3 alınabilir mi? Sonra 2-3'e bakılırsa ne olur?

> **Çözüm:** 0 alınmış ama 3 alınmamıştır, 0-3 alınabilir, döngü oluşmaz. Bundan sonra 0, 1, 2, 3 aynı alt graftadır. 2-3'e bakıldığında hem 2 hem 3 alınmış ve aynı alt graftadır; 2-3 alınırsa 0-1-2-3-0 döngüsü oluşur, alınmaz. Python ile aynı kenar sırası işletildiğinde de 0-1, 1-2, 0-3 alındı, 2-3 alınmadı.

Soru 6 (derste sözlü, 02:26:57)

İki ayrı alt graf oluşmuşken hem 3 hem 4 daha önce alınmış olsun. 3-4 alınmalı mı?

> **Çözüm:** Alınmalıdır. 3 ile 4 aynı alt grafta değildir; 3-4 kenarı iki ayrı alt grafı birbirine bağlar, döngü oluşturmaz. Uçların daha önce alınmış olması tek başına yetmez, aynı alt grafta olup olmadıklarına bakılır. Karşı örnek: 0-1, 1-2, 2-3 alınmışsa 0 ile 3 aynı alt graftadır ve 0-3 alınmaz.

Soru 7 (derste sözlü, 02:34:30)

Etiket dizisinde bir (u, v) kenarının iki ucunun etiketi de x ise ne yapılır?

> **Çözüm:** Kenar alınmaz (bağlanmaz). İki uç zaten aynı alt graftadır, kenar eklenirse döngü oluşur. Sınıftan bir öğrenci bu cevabı verdi.

Soru 8 (derste sözlü, 02:37:01)

n düğümlü bir grafın minimum yayılan ağacında kaç kenar olur?

> **Çözüm:** $n - 1$ kenar olur. İki düğüm için bir kenar, üç düğüm için iki kenar gerekir. Ağaca bir kenar daha eklenirse döngü oluşur. Örnek grafta 8 düğüm vardı ve hem Prim hem Kruskal 7 kenar seçti.

Soru 9 (derste sözlü, 02:39:18)

1-2 ve 3-4 kenarları seçilmiş olsun. 2-3 alınmalı mı? 2-3 alındıktan sonra 2-4 alınmalı mı?

> **Çözüm:** 2-3 alınır, çünkü {1, 2} ve {3, 4} iki farklı bağlı bileşendir ve 2-3 bunları birleştirir. 2-3 alındıktan sonra 1, 2, 3, 4 aynı bağlı bileşendedir; 2-4 alınırsa 2-3-4 döngüsü oluşur, alınmaz. Python ile aynı sıra işletildiğinde de 1-2, 3-4, 2-3 alındı, 2-4 alınmadı.

Dersin Toparlaması

Bu derste doğrusal veri yapılarından doğrusal olmayan graf veri yapısına geçildi. Grafın internet, haritalar, sosyal ağlar, iletişim ağları ve devreler gibi kullanım alanları görüldü. Ardından terimler tanımlandı: düğüm, kenar, komşuluk, derece, yol, döngü, bağlı bileşen, ağaç, yayılan ağaç, klik, bağlı ve bağlı olmayan graf, iki parçalı graf ve ağırlıklı graf. Kodlamada düğümlerin sayılarla temsil edildiği, isimlerin bir sembol tablosunda tutulduğu anlatıldı. Grafı saklamanın iki yolu karşılaştırıldı: komşuluk matrisi V^2 yer tutar ve kenar sorgusunu sabit zamanda yapar; komşuluk listesi $V + E$ yer tutar ve seyrek graflar için uygundur. Komşuluk listesinin C'deki gerçeklemesi, başa ekleme mantığıyla adım adım izlendi. Yönlü graflarda matrisin simetrik olmak zorunda olmadığı, derecenin giriş ve çıkış derecesine ayrıldığı görüldü.

Dersin ikinci yarısında minimum yayılan ağaç tanımlandı ve aynı 8 düğümlü graf üzerinde Prim ve Kruskal algoritmaları işletildi. İki algoritma da toplam ağırlığı 204 olan aynı ağacı buldu. Kruskal'da döngü kontrolünün, her düğümün hangi alt grafa ait olduğunu tutan bir etiket dizisiyle yapılabileceği gösterildi. n düğümlü bir MST'de $n-1$ kenar bulunduğu hatırlatıldı. Karmaşıklık olarak matrisli Prim için $O(V^2)$, Kruskal için sıralamadan gelen $O(E \log V)$ verildi. Hoca, Kruskal algoritmasının kodunu bir sonraki derste yazacaklarını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.